DRONACHARYA
College of Engineering

SMART AI FACIAL SKIN CARE ASSISTANT

PROJECT II [P.22.6.631]

SUBMITTED IN PARTIAL FULFILLMENT OF THE

REQUIREMENTS FOR THE AWARD

OF

DEGREE OF BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE & ENGINEERING

(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

Gurugram University, Gurugram

Submitted By

Akshit Kaushik

University Roll No. 120181437

University Reg. No. 231371360002

Department of Computer Science & Engineering (AI&ML)

DRONACHARYA COLLEGE OF ENGINEERING

KHENTAWAS, GURGAON, HARYANA

DRONACHARYA
College of Engineering

PROJECT II

[P.22.6.631]

SMART AI FACIAL SKIN CARE ASSISTANT

SUBMITTED IN PARTIAL FULFILLMENT OF THE

REQUIREMENTS FOR THE AWARD

OF

DEGREE OF BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE & ENGINEERING

(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

Submitted By

Akshit Kaushik

University Roll No. 120181437

University Reg. No. 231371360002

Department of Computer Science & Engineering (AI&ML)

AFFIILIATED TO

Gurugram University, Gurugram, HARYANA

ACADEMIC SESSION: 2024 - 2026

DRONACHARYA
College of Engineering

CERTIFICATE

This is to certify that the project report entitled "***Smart AI Facial skin care Assistant***" submitted by "***Akshit Kaushik (25921)***" in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science & Engineering (AI&ML)** of Dronacharya College of Engineering, Gurugram is a record of bonafide work carried out under my guidance and supervision.

Dr. Sandeep ***Prof. Sanjyoti***

Department of Computer Science & Engineering (AI&ML)

Dr. Ritu Pahwa

(Head of Department)
(Signature and Seal)

DRONACHARYA
College of Engineering

STUDENT DECLARATION

I,**AKSHIT KAUSHIK**, of 7TH semester Btech (CSE AI&ML), in the Department of Bachelor of Technology in Computer Science & Engineering from Dronacharya College of Engineering, Gurugram, hereby declare that the project work entitled " **SMART AI FACIAL SKIN CARE ASSISTANT** " is carried out by us and submitted in partial fulfillment of the requirements for the award of **Bachelor of Technology in Computer Science & Engineering (AI&ML)** , under **Dronacharya College of Engineering** during the academic year 2024 - 2025 and has not been submitted to any other university for the award of any kind of degree.

Place:-

Date:

……..

(Student Signature)

Akshit Kaushik

Roll No.: 25921

University Roll No.: 120181437

University Registration No.: 231371360002

DRONACHARYA
College of Engineering

ACKNOWLEDGEMENT

The success and final outcome of this project require a lot of guidance and assistance from many people and I am extremely privileged to have got this all along the completion of my project. All that I have done is only due to such supervision and assistance and I would not forget to think them. I respect and thank **Dr. Ritu Pahwa, HOD CSE (AI&ML),** Dronacharya College of Engineering, Affiliated to Gurugram University, Gurugram for providing me an opportunity to do the project work. I am extremely thankful to her for providing such nice support and motivation. I owe my deep gratitude to our project mentors **Dr. Sandeep and Prof. Sanjyoti** who took keen interest in our project work and guided us all along, till the completion of our project work by providing all the necessary information for developing a good system.

……..

(Student Signature)

Date:

Akshit Kaushik

Roll No.: 25921

University Roll No.: 120181437

University Registration No.: 231371360002

TABLE OF CONTENTS

LIST OF FIGURES

CHAPTER 1
INTRODUCTION TO THE PROJECT

1.1 Introduction

The *AI Smart Skin Care System* is an intelligent web-based application that analyzes a person's facial skin image to determine their skin type (Oily, Dry, or Normal) using Deep Learning techniques.
This project aims to bridge the gap between dermatology and technology by providing accurate skin-type detection and personalized skin care recommendations.

The system uses a Convolutional Neural Network (CNN) model trained on categorized images of different skin types. It then predicts the skin type from a captured or uploaded image in real time and provides a complete recommendation report containing SPF level, moisturizer type, water intake suggestions, and skincare tips for both summer and winter seasons.

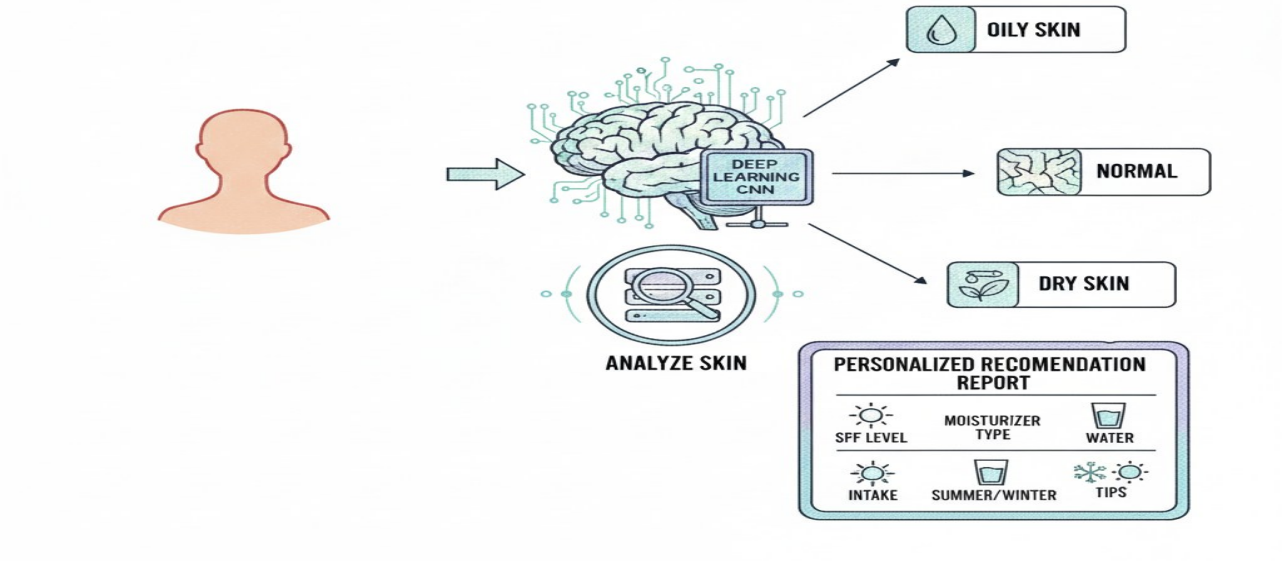

Figure 1.1 WorkFlow

1.2 Background

Skin is the largest organ of the human body, and its health plays a vital role in overall well-being. Improper care or lack of knowledge about one's skin type can lead to acne, dryness, or excessive oiliness. Traditionally, identifying skin type requires dermatologist consultation, which is often time-consuming and expensive.

With advancements in **Artificial Intelligence (AI)** and **Machine Learning (ML)**, image-based skin analysis has become feasible and reliable. AI models can process thousands of facial samples, identify subtle patterns, and classify skin conditions effectively.

This project leverages **Computer Vision** and **Deep Learning (CNN)** techniques to automate this process and provide real-time, accurate, and cost-effective skin-type detection.

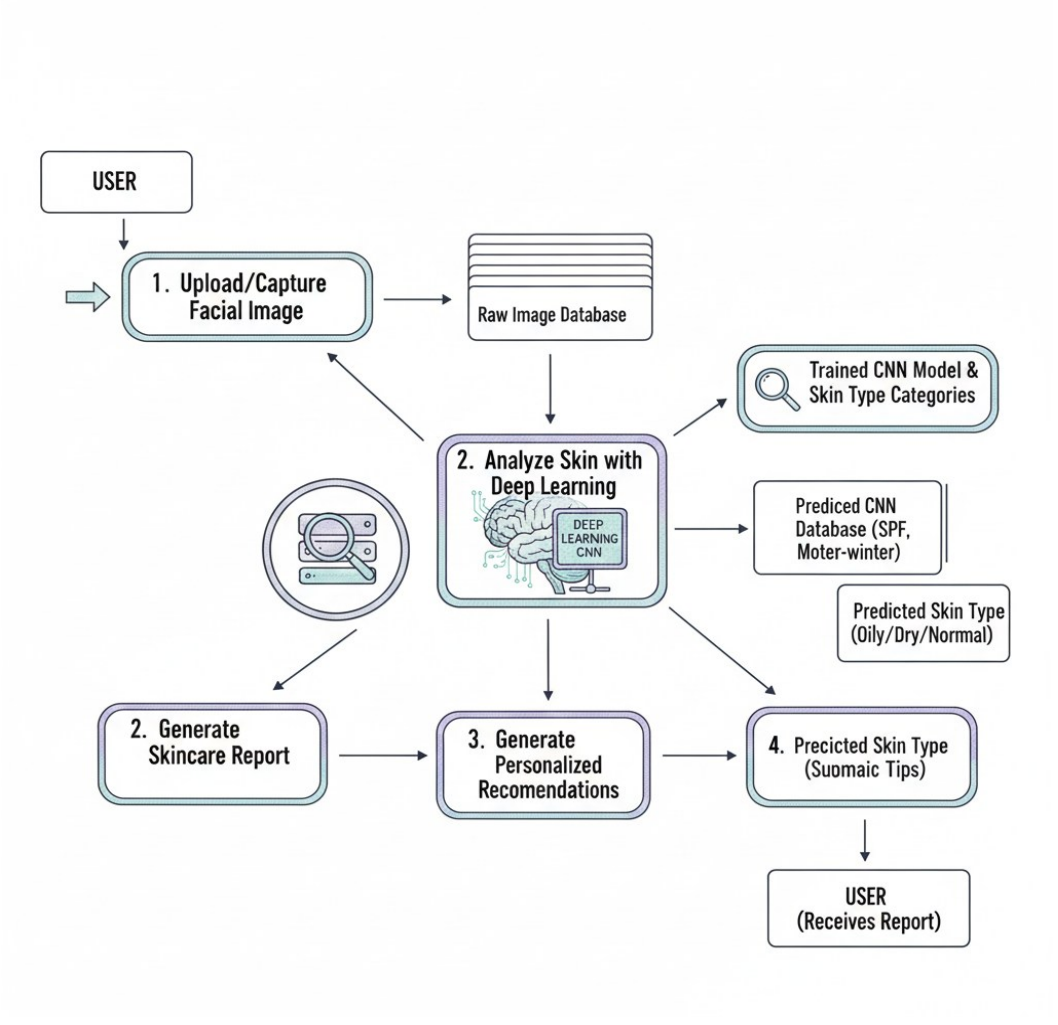

Figure 1.2 Data Flow

1.3 Objectives

The main objective of the AI Smart Skin Care System is to develop an intelligent and automated method for analyzing facial skin and identifying the user's skin type accurately using deep learning techniques.

The system aims to simplify skincare assessment, provide personalized recommendations, and bridge the gap between dermatology and modern AI technology.

Objectives of this project are:

1. To develop a deep learning model capable of classifying skin types accurately (Oily, Dry, Normal).
2. To design an interactive web interface for real-time skin analysis using **Streamlit**.
3. To generate user-friendly reports containing skincare recommendations based on predicted skin type.
4. To store and maintain user analysis history in a structured database (CSV format).
5. To enhance awareness of personalized skincare through AI-driven technology.
6. Specific Objectives:
    - To build a **CNN-based deep learning model** capable of classifying skin types (oily, dry, normal) from facial images.
    - To create a **user-friendly interface** using Streamlit that allows users to upload images and receive instant results.
    - To develop a system that generates a **personalized skincare recommendation report** based on the predicted skin type.
    - To use Pandas and NumPy for **efficient data handling, preprocessing, and analysis** during model development.
    - To integrate ReportLab for **professional PDF report generation** containing user-specific skincare guidance.
    - To enhance accessibility by providing a **fast, accurate, and automated solution** that does not require expert dermatological knowledge.

1.4 Suitability of the project

The AI Smart Skin Care System is highly suitable for modern-day skincare needs due to the increasing demand for personalized, accurate, and accessible dermatological guidance.

As many individuals struggle to correctly identify their skin type and select appropriate skincare products, this system provides an intelligent and convenient solution.

The project fits well within the growing intersection of artificial intelligence, healthcare, and consumer technology, making it relevant, practical, and impactful.

- **Reasons for Suitability**
- The project addresses a **real-world problem** where people often misjudge their skin type, leading to incorrect product usage and skin issues.
- It uses **deep learning and image processing**, which are ideal technologies for analyzing skin patterns and textures accurately.
- The system provides **instant results** through a Streamlit web interface, making it suitable for everyday users without technical knowledge.
- It is suitable for both **individual users and dermatology clinics**, offering quick pre-assessment or self-check tools.
- The project is highly adaptable and can be **extended to detect complex skin conditions** such as acne, pigmentation, or wrinkles.
- It can run on **local machines or cloud platforms**, making it suitable for a variety of deployment environments.
- The lightweight design ensures it is suitable even for **mobile or low-power devices** with minor optimizations.
- It promotes **personalized skincare**, which is becoming a major trend in the beauty and healthcare industries.
- **Overall Suitability**

The project is well-suited to meet current market and healthcare demands by offering an AI-driven, accurate, easy-to-use, and scalable skin analysis system. Its combination of useful technologies and practical application makes it not only feasible but highly valuable for real-world deployment.

1.5 Significance of the Project

- This project contributes to the integration of Artificial Intelligence into dermatological applications.
  It simplifies skincare analysis and provides instant results without expert supervision. Users can perform a self-assessment anytime, anywhere using their mobile or webcam.
- The system also promotes:
- Awareness of daily skincare habits.
- Preventive care against skin issues caused by oil imbalance or dehydration.
- A foundation for advanced AI dermatology systems capable of detecting acne, pigmentation, and other conditions.
- By combining technology and healthcare, this project represents a step forward toward **AI-based personal wellness solutions**.
- **Why This Project Is Significant:**
  - It offers an **automated and reliable method** to identify skin types without requiring dermatologists or manual testing.
  - It makes skincare guidance **accessible to everyone**, especially in remote or underserved areas.
  - It helps users avoid **incorrect products** by giving personalized recommendations based on AI prediction.
  - It promotes **early skin care awareness**, reducing the chances of long-term skin damage.
  - It demonstrates the real-world application of **deep learning, image processing, and AI** in healthcare.
  - It provides a foundation for developing **advanced dermatology tools** that can detect complex skin conditions in the future.
  - It integrates multiple technologies (CNN, Streamlit, ReportLab, Pandas, NumPy), showcasing a complete end-to-end AI solution.

1.6 Methodology

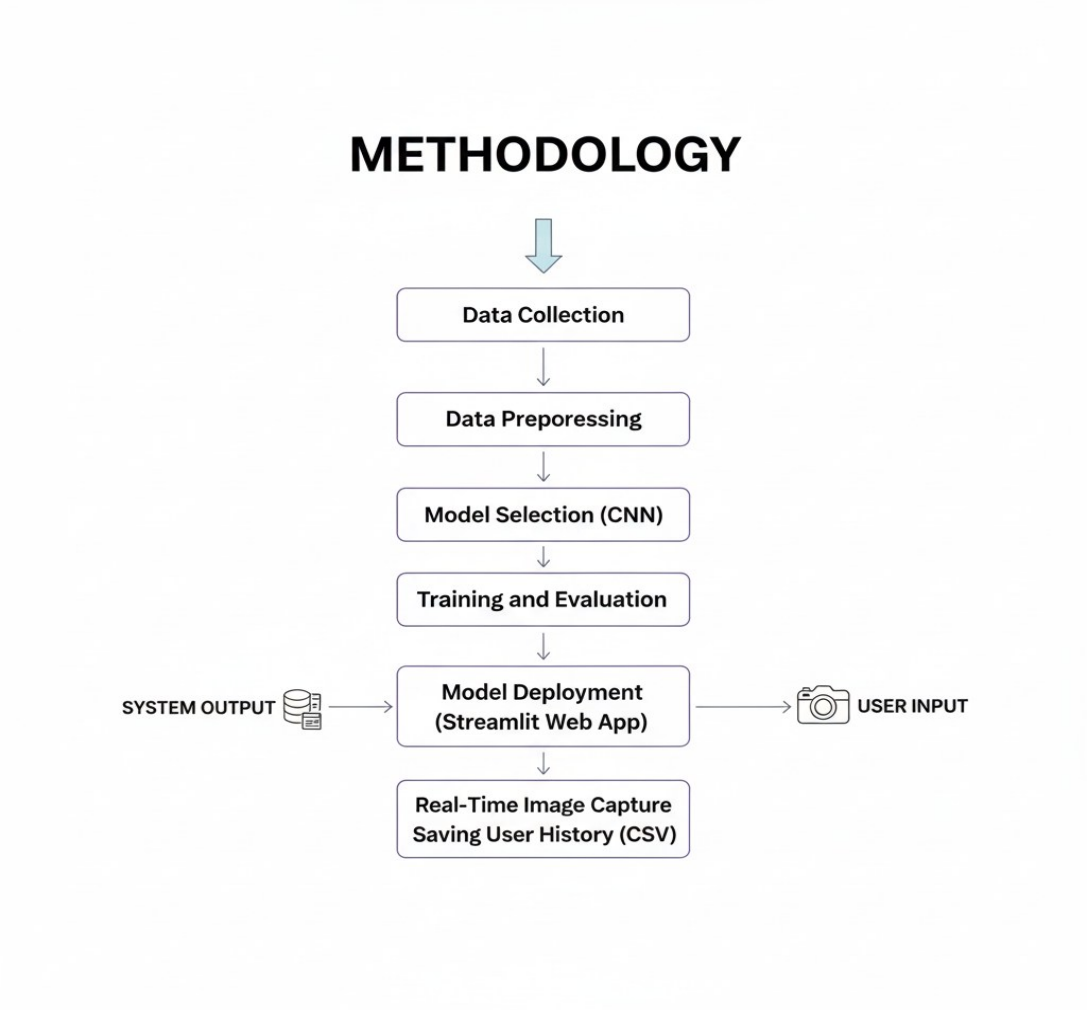

Figure 1.3 Methodology

1.6.1 Data Collection

A dataset of facial images categorized as **Oily**, **Dry**, and **Normal** was collected from open source platforms such as Kaggle.
The data was divided into:

- **Training Set (70%)**
- **Validation Set (20%)**
- **Testing Set (10%)**

1.6.2 Data Preprocessing

Images were resized to **128×128**, normalized (pixel values scaled between 0–1), and augmented using random rotation, zoom, and flip.
This helped improve model generalization and reduce overfitting.

1.6.3 Model Selection

A **Convolutional Neural Network (CNN)** was designed using **TensorFlow and Keras** with:

- Convolution + Pooling layers for feature extraction
- Dense layers for classification
- Softmax output for 3 skin types (Oily, Dry, Normal)

Optimizer: **Adam**, Loss: **Categorical Crossentropy**, Epochs: **15**

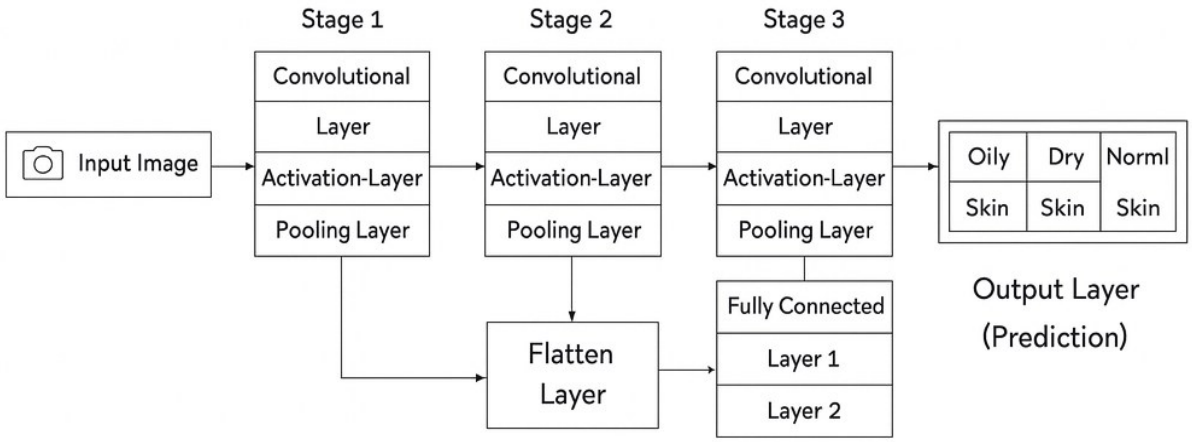

Figure 1.4 CNN(1)

1.6.4 Training and Evaluation

- The model was trained on the pre-processed dataset, where each image was normalized, resized, and augmented to improve generalization.
- During training, the dataset was divided into training and validation sets, allowing the model to learn from one portion while being tested on unseen images from the validation set. This helped monitor how well the model generalized beyond the data it was trained on. The performance of the CNN was evaluated using accuracy and loss curves, which were plotted after each epoch. These curves made it possible to observe trends such as improvement in learning, signs of underfitting or overfitting, and overall model stability.
- A steadily increasing validation accuracy and decreasing validation loss indicated that the model was learning robust features and performing reliably on unseen data.

1.6.5 Deployment

The deployment of the AI Smart Skin Care System is carried out using **Streamlit**, an open-source Python framework designed for creating interactive and easy-to-use web applications. Streamlit allows the entire machine learning model, image-processing pipeline, and recommendation system to be accessed through a simple browser interface without requiring users to install any additional software. This makes the system highly accessible, user-friendly, and practical for real-world usage.

The trained CNN model is integrated with Streamlit so that users can upload a skin image, process it instantly, and receive accurate skin-type predictions along with personalized skincare recommendations. The deployment focuses on providing a smooth and responsive user experience while ensuring that the model runs efficiently on local or cloud environments.

- **Deployment Process Overview**
- The trained model (.h5 or .pt file) is loaded inside the streamlit_app.py application.
- A clean user interface is designed using Streamlit widgets such as st.file_uploader(), st.image(), and st.button().
- When an image is uploaded, the app processes it, runs it through the CNN model, and displays the predicted skin type.
- Personalized recommendations are generated dynamically and shown on the screen.
- ReportLab is integrated for PDF report generation, allowing users to download customized skincare reports.
- The application is executed locally using the terminal command: **streamlit run streamlit_app.py**

1.6.6 Capture and Prediction

The project provides two ways for users to input their images:

1. **Capture via Webcam**:
    - Using **OpenCV**, the user can take a live photo for skin analysis.
    - The captured image is preprocessed to match the input shape of the TensorFlow model.
    - The model predicts the skin type (Oily, Dry, Normal) in real-time.
2. **Upload Image Option**:
    - Users can upload an existing photo from their device using **Streamlit's file uploader**.
    - The uploaded image is converted into an array and preprocessed similarly to webcam input.
    - The prediction is displayed along with seasonal recommendations.
3. **Prediction:**

    After capturing or uploading an image, the system preprocesses it to match the input requirements of the TensorFlow model. This preprocessing involves resizing the image to the correct dimensions and normalizing the pixel values to a 0–1 range. A batch dimension is added to make the image compatible with the model. The preprocessed image is then passed into the trained TensorFlow model, which outputs probabilities for each skin type class: Oily, Dry, or Normal. The class with the highest probability is selected as the predicted skin type. Finally, the predicted skin type is displayed to the user along with seasonal recommendations or personalized advice, providing accurate and immediate feedback based on the user's skin image.

After analysis, the app generates a **PDF report** summarizing:

- Predicted skin type
- Oiliness/Dryness rate
- SPF and moisturizer suggestions
- Seasonal tips (Summer/Winter)

User results are also saved in a **CSV file** for history tracking.

1.6.7 Saving User History (CSV)

- In this project, the results of each skin analysis are stored in a CSV file.
- The CSV file maintains a history of user analyses for future comparison.
- Each entry includes the date and time of the analysis.
- The user name is also stored, allowing multiple users to maintain their records separately.
- The predicted skin type (Oily, Dry, Normal) is saved for tracking skin condition over time.
- Seasonal recommendations provided by the system are included in the CSV.
- Additional personalized notes or advice are recorded to help users follow tailored skincare routines.
- The CSV is managed using Python's pandas library, which enables easy creation, reading, and appending of data.
- Whenever a user performs a new analysis, their data is appended to the CSV without overwriting previous records.
- This ensures a complete history of all analyses is maintained for each user.
- Saving results in CSV also provides a backup for PDF reports generated by the system.
- It enables tracking and comparison of skin health over time, helping users make informed skincare decisions.

1.6.8 Maintenance

- Model performance can be enhanced through regular retraining using more diverse and updated datasets. Continuous monitoring of prediction accuracy, removing outdated data, and fine-tuning parameters also help maintain reliability.
- Additionally, incorporating user feedback and system logs enables the model to adapt to new patterns, leading to better recommendations over time.

CHAPTER 2

TOOLS AND TECHNOLOGY

2.1 Python

- Python is the primary programming language used for developing the entire system.
- It provides powerful libraries for machine learning and deep learning, which are essential for building the CNN model.
- TensorFlow and Keras were used to design, train, and evaluate the skin-type classification model.
- OpenCV helped in image preprocessing tasks such as resizing, normalization, and face analysis.
- NumPy and Pandas were used for efficient data handling, processing, and manipulation.
- Python also supports easy integration with web frameworks like Flask or Streamlit for creating the user interface.
- Its simple syntax allows faster debugging and rapid prototyping of AI models.
- Strong community support and continuous library updates make Python reliable for long-term project development.
- Overall, Python's flexibility and rich ecosystem make it highly useful and effective for implementing this AI-based skin analysis project.

2.2 TensorFlow

- IT is an open-source deep learning framework developed by Google.
- TensorFlow is a powerful deep learning framework used as the backend for training the CNN model.
- It handles large numerical computations efficiently and supports GPU acceleration for faster training.
- In this project, TensorFlow works with Keras to perform operations like convolution, backpropagation, and optimization.
- Overall, TensorFlow ensures fast, stable, and reliable execution of the skin-type prediction system.

2.3 Keras

- Keras is a high-level deep learning framework used in this project to build and train the Convolutional Neural Network (CNN).
- It provides simple and user-friendly APIs that make designing neural network architectures fast and intuitive.
- Keras runs on top of TensorFlow, allowing access to powerful GPU-accelerated computations for faster training.
- In this project, Keras is used to define layers such as convolution, pooling, flattening, and fully connected layers.
- It also handles model compilation, where the loss function, optimizer, and evaluation metrics are specified.
- Keras makes training easier by providing built-in functions for fitting the model, tracking accuracy, and generating training/validation curves.
- Its flexibility allows quick experimentation with different model architectures and hyperparameters.

2.4 OpenCV

- OpenCV (Open Source Computer Vision Library) is widely used for image processing and computer vision tasks.
- It provides fast and efficient functions for handling images, videos, and real-time camera input.
- In this project, OpenCV is used to preprocess skin images before sending them to the CNN model.
- Tasks such as resizing, cropping, normalization, and noise reduction are performed using OpenCV.
- It helps detect the face region and extract relevant skin features accurately.
- OpenCV supports various image transformations that improve the quality of input data.
- The library is optimized for performance, allowing faster processing even for large datasets.
- Overall, OpenCV plays an important role in producing clean and consistent input images, which improves the model's prediction accuracy.

2.5 Pandas

- It is used in this project for organizing, storing, and managing user history in a structured format.
- It allows easy handling of CSV files, making it convenient to save user inputs, predictions, and system-generated recommendations.
- Pandas is a Python library that helps you work with data in rows and columns.
  In the AI Smart Skin Care System, you first use Pandas to load your skin-image dataset from a CSV file.
- Once loaded, Pandas lets you clean the data by removing missing or incorrect entries.
  It helps you convert skin-type labels like oily, dry, and normal into numeric values for the CNN model.
- Pandas also helps organize image paths and labels so the model knows which image belongs to which category.
- You use Pandas to split the dataset into training and testing sets before training the CNN.
- After the model predicts a skin type, Pandas stores those predictions in a structured table.
- This stored data is then used to generate a personalized skincare recommendation report.
- Pandas can also track multiple users' results and allow you to analyze overall patterns in the skin data..

2.6 NumPy

- It is used for fast numerical computations required during image processing and model preparation.
- It efficiently handles arrays and matrices, which are essential for representing image data and feeding it into the CNN model.
- NumPy's optimized mathematical operations make data preprocessing and model computations faster and more reliable.
- In this AI skin-care project, NumPy helps handle image data because images are stored as numeric pixel arrays.

2.7 ReportLab

- ReportLab is a Python library used to create PDF files directly from your code.
- In your AI skin-care system, ReportLab helps you generate a professional PDF report for each user.
- When the model predicts the user's skin type, ReportLab lets you place that result inside a clean PDF layout.
- It allows you to add text, headings, and paragraphs describing skincare recommendations. You can also add tables, sections, or styled content to make the report look organized.
- ReportLab supports adding images, so you can include the user's analyzed skin image if needed.
- The library gives you full control over fonts, spacing, and formatting, making the PDF look polished.
- Finally, ReportLab saves the report as a downloadable PDF that the user can open, share, or print.

2.8 Streamlit

- Streamlit is a Python library used to build interactive web apps quickly and easily.
- In your AI skin-care project, Streamlit helps you create a user interface without needing HTML or JavaScript.
- It allows users to upload their skin images directly through the web app.
- Once they upload an image, Streamlit displays it on the screen so the user can confirm it.
- Streamlit then sends the image to your CNN model for skin-type prediction.
- After the prediction, Streamlit shows the result—like oily, dry, or normal—instantly on the page.
- You can also use Streamlit to display personalized skincare recommendations in a clean layout.
- If you generate a PDF report (using ReportLab), Streamlit can provide a button for the user to download it.
- Streamlit refreshes everything in real time, making the app feel smooth and interactive.
- Overall, Streamlit turns your machine-learning code into a complete, user-friendly web application.

CHAPTER 3

SNAPSHOTS

```
# train_model.py
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.optimizers import Adam
import os

train_dir = r"C:\Users\pc\Desktop\project_7_sem\AI_Smart_Skin_Care\Oily-Dry-Skin-Types\train"
val_dir = r"C:\Users\pc\Desktop\project_7_sem\AI_Smart_Skin_Care\Oily-Dry-Skin-Types\valid"

# Image generators
train_datagen = ImageDataGenerator(rescale=1./255, rotation_range=20, zoom_range=0.2, shear_range=0.2, horizontal_flip=True)
val_datagen = ImageDataGenerator(rescale=1./255)

train_gen = train_datagen.flow_from_directory(train_dir, target_size=(128, 128), batch_size=32, class_mode='categorical')
val_gen = val_datagen.flow_from_directory(val_dir, target_size=(128, 128), batch_size=32, class_mode='categorical')

# CNN model
model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)),
    MaxPooling2D(2,2),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2,2),
    Conv2D(128, (3,3), activation='relu'),
    MaxPooling2D(2,2),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(3, activation='softmax')  # 3 classes: dry, oily, normal
])

model.compile(optimizer=Adam(learning_rate=0.0001), loss='categorical_crossentropy', metrics=['accuracy'])

history = model.fit(train_gen, validation_data=val_gen, epochs=15)

model.save("skin_type_model.h5")
print("✅ Model saved successfully as skin_type_model.h5")
```

Fig. 3.1 Source Code (1)

```
import streamlit as st
import pandas as pd
import numpy as np
import cv2
import os
import datetime
import base64
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing import image
from reportlab.lib.pagesizes import A4
from reportlab.pdfgen import canvas

# ---------------- PATHS ----------------
MODEL_PATH = "skin_type_model.h5"
CSV_FILE = "skin_analysis_history.csv"
OUTPUT_DIR = "outputs"
os.makedirs(OUTPUT_DIR, exist_ok=True)

# ---------------- LOAD MODEL ----------------
try:
    model = load_model(MODEL_PATH)
except Exception as e:
    st.error("⚠️ Could not load model. Train it first using train_model.py.")
    st.stop()

# ---------------- SKIN CARE DATA ----------------
skin_data = {
    ("Oily", "Summer"): {
        "Oiliness": "80-90%",
        "Dryness": "10-20%",
        "Water": "2.5-3.0 L/day",
        "SPF": "SPF 40-50 (gel-based, non-comedogenic)",
        "Moisturizer": "⭐⭐⭐ (3/5)",
        "Type": "Light gel / water-based",
        "Tips": "Wash face 2-3 times daily, avoid heavy creams"
    },
    ("Oily", "Winter"): {
        "Oiliness": "70-80%",
        "Dryness": "20-30%",
        "Water": "2.5-3.0 L/day",
        "SPF": "SPF 30-40",
        "Moisturizer": "⭐⭐⭐⭐ (4/5)",
        "Type": "Lightweight lotion with hyaluronic acid",
        "Tips": "Use mild cleanser, hydrate skin daily"
    },
    ("Dry", "Summer"): {
        "Oiliness": "20-30%",
        "Dryness": "70-80%",
        "Water": "3.0-3.5 L/day"
```

Ln 246, Col 33 | 10,356 characters | Plain text

Fig. 3.2 Source Code (2)

```
# ---------------- FUNCTIONS ----------------
def predict_skin_type(img_path):
    img = image.load_img(img_path, target_size=(128, 128))
    img_arr = image.img_to_array(img) / 255.0
    img_arr = np.expand_dims(img_arr, axis=0)
    preds = model.predict(img_arr)
    labels = ["Dry", "Normal", "Oily"]
    return labels[np.argmax(preds)], float(np.max(preds) * 100)

def create_pdf_report(user_name, skin_type, analyzed_path, rec):
    from reportlab.lib import colors
    from reportlab.lib.units import inch

    pdf_path = os.path.join(OUTPUT_DIR, f"{user_name}_Skin_Report.pdf")
    c = canvas.Canvas(pdf_path, pagesize=A4)
    width, height = A4

    # ----------------- HEADER -----------------
    c.setFont("Helvetica-Bold", 20)
    c.setFillColor(colors.darkgreen)
    c.drawCentredString(width / 2, height - 60, "🌸 AI Smart Skin Care Report 🌸")
    c.setFillColor(colors.black)
    c.line(40, height - 70, width - 40, height - 70)

    # ----------------- DETAILS -----------------
    c.setFont("Helvetica", 12)
    y = height - 110
    line_space = 20

    info_lines = [
        f"👤 User Name: {user_name}",
        f"🧠 Predicted Skin Type: {skin_type}",
        f"💧 Oiliness: {rec['Oiliness']}    🌵 Dryness: {rec['Dryness']}",
        f"🥤 Recommended Water Intake: {rec['Water']}",
        f"😎 SPF Recommendation: {rec['SPF']}",
        f"🧴 Moisturizer: {rec['Moisturizer']} ({rec['Type']})",
        "",
        f"💡 Tips:",
        f"{rec['Tips']}"
    ]

    for line in info_lines:
        c.drawString(60, y, line)
        y -= line_space
        if y < 200:  # Avoid overlap
            c.showPage()
            c.setFont("Helvetica", 12)
            y = height - 100
```

Fig. 3.3 Source Code (3)

```
                """, unsafe_allow_html=True
            )

            st.markdown(
                f"""
                <div style="background:#e7f7ee; border-left:6px solid #4CAF50; border-radius:10px; padding:15px; margin-top:20px;">
                    <p style="font-size:15px; color:#2E8B57; margin:0;"><b>💡 Skin Care Tip:</b> {rec['Tips']}</p>
                </div>
                """, unsafe_allow_html=True
            )

            col1, col2 = st.columns(2)
            with col1:
                with open(pdf_path, "rb") as f:
                    st.download_button("📄 Download PDF Report", f, file_name=os.path.basename(pdf_path))
            with col2:
                with open(analyzed_path, "rb") as f:
                    st.download_button("🖼️ Download Analyzed Image", f, file_name=os.path.basename(analyzed_path))

# ================== VIEW HISTORY ==================
elif choice == "📂 View Past Reports":
    st.subheader("📜 Your Saved Skin Analysis History")
    if os.path.exists(CSV_FILE):
        try:
            df = pd.read_csv(CSV_FILE)
            st.dataframe(df)
        except Exception:
            st.warning("⚠️ History file is corrupted. Delete it and try again.")
    else:
        st.info("No reports found yet. Please analyze your skin first.")

# ================== ABOUT ==================
else:
    st.subheader("ℹ️ About Project")
    st.write("""
    **AI Smart Skin Care** is an ML-powered application that:
    - Predicts your **skin type** (Oily, Dry, Normal)
    - Provides **seasonal recommendations**
    - Generates a **personalized PDF report**
    - Saves your **history** for future comparison
    Built using **TensorFlow, OpenCV, and Streamlit** 💻
    """)
```

Fig. 3.4 Source Code (4)

```
        "Confidence": conf,
        "Season": season,
        "Oiliness": rec["Oiliness"],
        "Dryness": rec["Dryness"],
        "Water": rec["Water"],
        "SPF": rec["SPF"],
        "Moisturizer": rec["Moisturizer"],
        "Type": rec["Type"],
        "Tips": rec["Tips"],
        "Date": datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    }
    df_new = pd.DataFrame([new_entry])
    if os.path.exists(CSV_FILE):
        df_old = pd.read_csv(CSV_FILE)
        df = pd.concat([df_old, df_new], ignore_index=True)
    else:
        df = df_new
    df.to_csv(CSV_FILE, index=False)

    # Create PDF
    pdf_path = create_pdf_report(user_name, result, analyzed_path, rec)

    # --- Stylish Result Display ---
    st.markdown("<h3 style='text-align:center; color:#4CAF50;'>✅ Skin Analysis Result</h3>", unsafe_allow_html=True)
    st.markdown(
        f"""
        <div style="text-align:center;">
            <img src="{base64.b64encode(open(analyzed_path,'rb').read()).decode()}"
                 style="border-radius:20px; box-shadow:0 4px 15px rgba(0,0,0,0.3); width:350px; margin:10px;">
        </div>
        """, unsafe_allow_html=True
    )

    st.markdown(
        f"""
        <div style="background:linear-gradient(135deg, #f9f9f9, #eafbea); border-radius:15px; padding:20px;
                    box-shadow:0 3px 12px rgba(0,0,0,0.1); margin-top:15px;">
            <h4 style="color:#2E8B57;">🌿 Predicted Skin Type: <b>{result}</b></h4>
            <p style="font-size:16px; color:#333;">🧠 <b>Confidence:</b> {conf:.2f}%</p>
            <hr>
            <p style="font-size:15px;">💧 <b>Oiliness:</b> {rec['Oiliness']}    |   
```

Fig. 3.5 Source Code (5)

```
# ---------------- SKIN CARE DATA ----------------
skin_data = {
    ("Oily", "Summer"): {
        "Oiliness": "80–90%",
        "Dryness": "10–20%",
        "Water": "2.5–3.0 L/day",
        "SPF": "SPF 40–50 (gel-based, non-comedogenic)",
        "Moisturizer": "⭐⭐⭐ (3/5)",
        "Type": "Light gel / water-based",
        "Tips": "Wash face 2–3 times daily, avoid heavy creams"
    },
    ("Oily", "Winter"): {
        "Oiliness": "70–80%",
        "Dryness": "20–30%",
        "Water": "2.5–3.0 L/day",
        "SPF": "SPF 30–40",
        "Moisturizer": "⭐⭐⭐⭐ (4/5)",
        "Type": "Lightweight lotion with hyaluronic acid",
        "Tips": "Use mild cleanser, hydrate skin daily"
    },
    ("Dry", "Summer"): {
        "Oiliness": "20–30%",
        "Dryness": "70–80%",
        "Water": "3.0–3.5 L/day",
        "SPF": "SPF 15–30 (cream-based)",
        "Moisturizer": "⭐⭐⭐⭐ (4/5)",
        "Type": "Hydrating cream with aloe vera",
        "Tips": "Avoid hot showers, apply moisturizer twice daily"
    },
    ("Dry", "Winter"): {
        "Oiliness": "10–20%",
        "Dryness": "80–90%",
        "Water": "3.5–4.0 L/day",
        "SPF": "SPF 20–30",
        "Moisturizer": "⭐⭐⭐⭐⭐ (5/5)",
        "Type": "Deep hydrating (shea butter, ceramides)",
        "Tips": "Use humidifier indoors, avoid alcohol-based toner"
```

Ln 266, Col 1 | 10,356 characters | Plain text

Fig. 3.6 Source Code (5)

CHAPTER 4

RESULTS

4.1 Skin analyzer

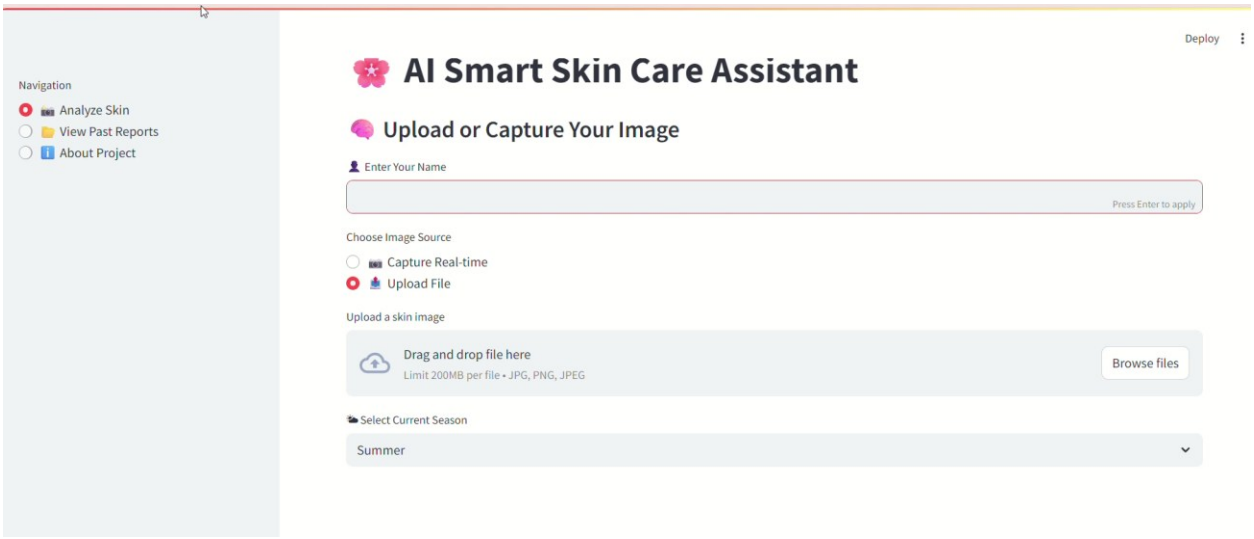

Fig. 4.1 Result Report (1)

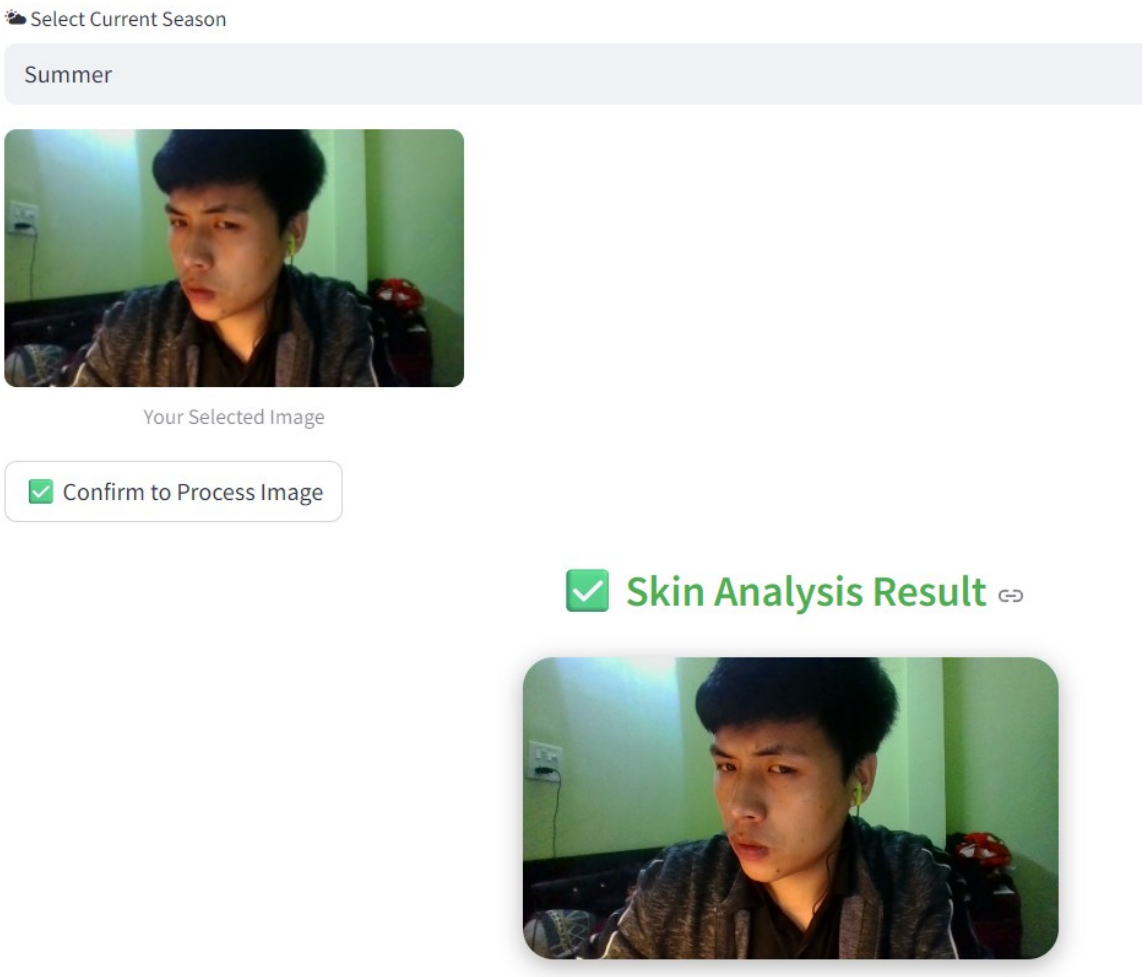

Fig. 4.1 Result Report (2)

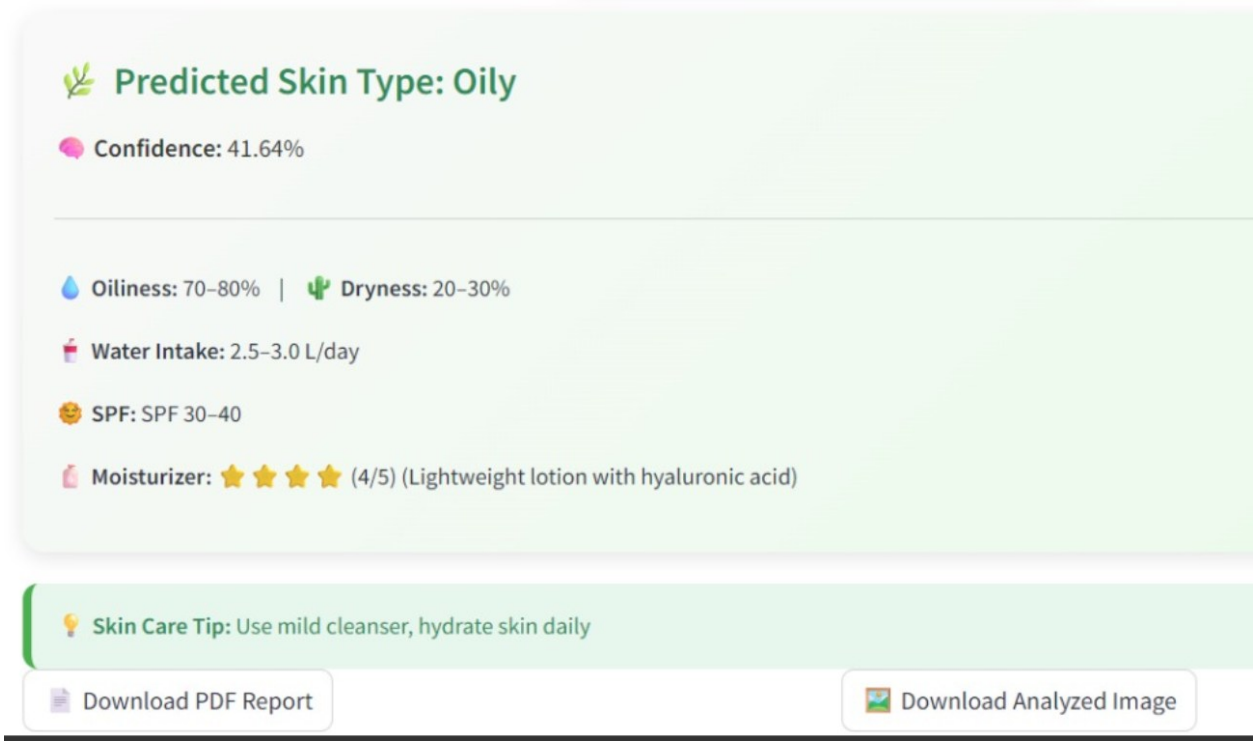

Fig. 4.2 Result report(3)

📜 Your Saved Skin Analysis History

	Name	SkinType	Confidence	Season	Oiliness	Dryness	Water	SPF
0	AKshit	Oily	43.162	Winter	70–80%	20–30%	2.5–3.0 L/day	SPF 30–40
1	Avinash	Normal	39.5919	Summer	80–90%	10–20%	2.5–3.0 L/day	SPF 40–50 (gel-based, non-comedogenic)
2	Avinash	Normal	37.7969	Winter	80–90%	10–20%	2.5–3.0 L/day	SPF 40–50 (gel-based, non-comedogenic)
3	Avinash	Normal	37.7969	Winter	80–90%	10–20%	2.5–3.0 L/day	SPF 40–50 (gel-based, non-comedogenic)
4	risita	Normal	37.7969	Winter	80–90%	10–20%	2.5–3.0 L/day	SPF 40–50 (gel-based, non-comedogenic)
5	Ricita Lamhenbi	Normal	37.7969	Winter	80–90%	10–20%	2.5–3.0 L/day	SPF 40–50 (gel-based, non-comedogenic)
6	Ricita Lamhenbi	Oily	41.6402	Winter	70–80%	20–30%	2.5–3.0 L/day	SPF 30–40
7	Avinash	Normal	41.2171	Winter	80–90%	10–20%	2.5–3.0 L/day	SPF 40–50 (gel-based, non-comedogenic)
8	rohit	Normal	42.2225	Winter	80–90%	10–20%	2.5–3.0 L/day	SPF 40–50 (gel-based, non-comedogenic)
9	rohit	Normal	38.9627	Winter	80–90%	10–20%	2.5–3.0 L/day	SPF 40–50 (gel-based, non-comedogenic)

		Moisturizer	Type	Tips	Date
0		⭐⭐⭐⭐ (4/5)	Lightweight lotion with hyaluronic acid	Use mild cleanser, hydrate skin daily	2025-11-02 00:24:23
1	(gel-based, non-comedogenic)	⭐⭐⭐ (3/5)	Light gel / water-based	Wash face 2–3 times daily, avoid heavy creams	2025-11-02 00:41:59
2	(gel-based, non-comedogenic)	⭐⭐⭐ (3/5)	Light gel / water-based	Wash face 2–3 times daily, avoid heavy creams	2025-11-02 01:00:58
3	(gel-based, non-comedogenic)	⭐⭐⭐ (3/5)	Light gel / water-based	Wash face 2–3 times daily, avoid heavy creams	2025-11-02 01:02:20
4	(gel-based, non-comedogenic)	⭐⭐⭐ (3/5)	Light gel / water-based	Wash face 2–3 times daily, avoid heavy creams	2025-11-02 01:02:57
5	(gel-based, non-comedogenic)	⭐⭐⭐ (3/5)	Light gel / water-based	Wash face 2–3 times daily, avoid heavy creams	2025-11-02 01:08:32
6		⭐⭐⭐⭐ (4/5)	Lightweight lotion with hyaluronic acid	Use mild cleanser, hydrate skin daily	2025-11-02 11:38:43
7	(gel-based, non-comedogenic)	⭐⭐⭐ (3/5)	Light gel / water-based	Wash face 2–3 times daily, avoid heavy creams	2025-11-03 09:37:22
8	(gel-based, non-comedogenic)	⭐⭐⭐ (3/5)	Light gel / water-based	Wash face 2–3 times daily, avoid heavy creams	2025-11-03 10:32:16
9	(gel-based, non-comedogenic)	⭐⭐⭐ (3/5)	Light gel / water-based	Wash face 2–3 times daily, avoid heavy creams	2025-11-03 10:33:01

Fig. 4.3 history saved (4)

■ AI Smart Skin Care Report ■

■ User Name: Ricita Lamhenbi
■ Predicted Skin Type: Normal
■ Oiliness: 80–90% ■ Dryness: 10–20%
■ Recommended Water Intake: 2.5–3.0 L/day
■ SPF Recommendation: SPF 40–50 (gel-based, non-comedogenic)
■ Moisturizer: ■■■ (3/5) (Light gel / water-based)

■ Tips:
Wash face 2–3 times daily, avoid heavy creams

Generated on 2025-11-02 01:08:32

Fig. 4.4 Result report PDF(4)

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

- The **AI Smart Skin Care System** successfully demonstrates how Artificial Intelligence can assist in personalized skincare analysis.
  By using a **Convolutional Neural Network (CNN)** trained on categorized facial images, the system accurately classifies a user's skin type as **Oily**, **Dry**, or **Normal**.
- The integration of **Streamlit** allows real-time image capture and instant analysis, while **ReportLab** provides professional PDF reports that summarize results, highlight detected areas, and recommend skincare routines.
  The system's user-friendly interface, accuracy, and portability make it an accessible solution for quick skin assessment without dermatologist intervention.
- This project proves that AI can meaningfully contribute to **digital dermatology** by saving time, minimizing cost, and promoting awareness of skin health and self-care practices.
- **Key Outcomes of the Project:**

- Automated detection of **oily, dry, and normal** skin types using CNN.
- Real-time skin analysis through an interactive **Streamlit web app**.
- Clean and efficient data handling using **Pandas and NumPy**.
- PDF report generation with personalized skincare advice using **ReportLab**.
- Easy user experience with fast predictions and clear recommendations.

This project successfully bridges **AI technology and skincare**, creating a practical solution that can assist users in understanding their skin better and taking personalized steps for improvement. The system lays a strong foundation for future dermatology-based AI applications.

5.2 Future Scope

Although the current system performs well for basic skin-type classification, it can be expanded and improved in several directions:

1. **Enhanced Dataset** — Incorporate larger, more diverse datasets covering different ethnicities, lighting conditions, and age groups for better generalization.
2. **Advanced Skin Condition Detection** — Extend capabilities to identify acne, pigmentation, dark spots, wrinkles, and other dermatological conditions.
3. **Cloud Deployment** — Host the model on **AWS**, **Azure**, or **Hugging Face Spaces** for global accessibility and mobile-friendly use.
4. **Personalized AI Recommendations** — Utilize Reinforcement Learning or Generative AI to suggest routines based on climate, lifestyle, and previous results.
5. **User Analytics Dashboard** — Add graphical history tracking (CSV → charts) to help users monitor skin changes over time.
6. **Security and Privacy** — Implement encryption and secure cloud storage to protect user images and personal data
7. **Possible Future Enhancements:**
    - **Detection of more skin conditions** such as acne, pigmentation, wrinkles, dark spots, sensitivity, redness, and pores.
    - **Use of advanced deep learning models** like EfficientNet, MobileNet, or Vision Transformers to improve accuracy.
    - **Building a mobile app** (Android/iOS) so users can access the system instantly through their phone camera.
    - **Integration with real-time camera feed** for live skin analysis instead of static image uploads.
    - **Personalized skincare routines** generated using user's age, gender, climate region, lifestyle, and previous analysis history.
    - **Recommendation of suitable products** by integrating skincare brands or dermatology databases.

CHAPTER 6
REFERENCES

References in IEE format

[1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Communications of the ACM*, vol. 60, no. 6, pp. 84–90, 2017.

[3] F. Chollet, "Keras: Deep learning library for Theano and TensorFlow," *GitHub repository*, 2015. [Online]. Available: https://keras.io

[4] M. Abuzneid and A. Mahmood, "Enhanced skin detection using CNN deep learning," *IEEE International Conference on Electro/Information Technology (EIT)*, pp. 025–031, 2018.

[5] P. Tschandl, C. Rosendahl, and H. Kittler, "The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions," *Scientific Data*, vol. 5, no. 1, 2018.

[6] M. O. Diagne, S. P. Singh, and N. A. Elshaikh, "AI-based smart skincare analysis using image processing and machine learning," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 12, no. 5, pp. 152–159, 2021.

[7] S. Kaur and R. Arora, "Skin disease detection using deep learning," *International Journal of Engineering and Advanced Technology (IJEAT)*, vol. 9, no. 3, pp. 540–544, 2020.

[8] A. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.

[9] A. VanderPlas, *Python Data Science Handbook: Essential Tools for Working with Data*, O'Reilly Media, 2016.

[10] A. Asokan and S. Nair, "Deployment of deep learning models using Streamlit for real-time analysis," *IEEE International Conference on Computational Intelligence and Data Science (ICCIDS)*, pp. 600–604, 2022.